Configurable Static Type Checking for Forth

Jaanus Pöial
Tallinn University of Technology

Abstract

Forth words exchange operands through a data stack, and Forth source may also change the parser and dictionary while it is being read. This paper describes a configurable source-level checker for that setting. The checker represents a phrase by the stack cells it requires and the cells it leaves. Each cell has a profile-defined type and may have a symbolic identifier that records value flow through words such as `DUP`, `SWAP`, and `OVER`.

Straight-line code is checked by matching the output of one word with the input of the next. Branches are combined only when the implementation can construct one common stack interface. Repetition uses a finite test that compares one pass with two passes. The paper presents each algorithm through source examples and pseudocode. The examples include unequal branch depths and a loop counterexample. The paper also distinguishes implemented checks from stronger guarantees. Normalization can discard value-flow information, and a single branch contract necessarily forgets path-specific alternatives. The loop test does not compute an inductive invariant.

The type hierarchy, primitive stack effects, parser words, defining words, literals, and control structures are supplied in text files. The native implementation is `gforth-evaluator.fs`. A complete artifact manifest and reproducible example are included. Comparisons with StrongForth, pluggable Forth typing, and WebAssembly show which restrictions belong to this prototype rather than to stack languages in general.

Keywords: Forth; static analysis; stack effect; type checking; symbolic execution; concatenative languages.

1 Introduction

A Forth *word* is a named operation. Operands normally come from the data stack instead of named parameters. This paper writes the top of stack on the right, following standard stack-effect notation:

```
+   ( n1 n2 -- n3 )
```

The effect says that `+` consumes two numbers and produces one. It constrains only the visible top part of the stack; cells below that part remain unchanged. A colon definition introduces a new word:

```
: SQUARED  ( n -- n*n ) DUP * ;
```

A conventional Forth system treats a stack effect as documentation. The comment may be missing, stale, or too weak to describe value flow. For example, the two results of `DUP` are copies of the same input. A checker also has to handle branches, loops, parser words, and words that extend the dictionary while source is processed.

The evaluator in this project reads three inputs:

1. a finite set of types, aliases, and subtype declarations;
2. word effects plus parser and control-word metadata;

3. the Forth source to check.

It infers effects for source phrases and colon definitions without executing the program. An earlier Java implementation established the basic framework [13]; the artifact studied here is the native Gforth version. The engine supports either permissive or restrictive profiles. A permissive profile primarily checks stack depth. A restrictive profile can distinguish numbers, flags, address classes, and execution tokens.

The checker addresses four tasks beyond tokenization:

- match adjacent stack interfaces in straight-line code;
- propagate types and value identities through stack shuffles;
- construct one interface for alternative control-flow paths;
- approximate repeated execution with a terminating local test.

These are design choices of this implementation. They are not limitations of concatenative languages as a class. WebAssembly, for example, validates loops against declared block interfaces, while StrongForth integrates typing into a Forth system. Section 11 compares these alternatives.

2 Analysis Model

2.1 Profile-defined types

A profile defines canonical type names, aliases, and subtype edges. A typical fragment is:

```
type X cell x
type n number N
type char c
type addr a
type c-addr ca
type a-addr aa

rel n < X
rel char < n
rel addr < X
rel c-addr < addr
rel a-addr < c-addr
```

Here `X`, `cell`, and `x` are aliases. The relation `a-addr < c-addr` means that an aligned address is acceptable where a character address is required. The loader calculates transitive subtype relationships, so it also knows that `a-addr` is an `addr` and an `X`.

When two stack cells meet at a composition boundary, the checker applies a simple rule:

- if the types are equal, keep that type;
- if one type is a subtype of the other, keep the more specific type;
- otherwise, report a type clash.

The checker does not search for a third common subtype when the two boundary types are unrelated. This keeps matching deterministic but is less expressive than full lattice-based subtyping.

2.2 Stack effects and value identifiers

A stack effect has an input side and an output side:

```
( input cells -- output cells )
```

An arbitrary deeper prefix passes through unchanged. For example, `( n - n )` can run on a stack containing other cells below the top number. Type-system literature sometimes calls that arbitrary prefix a *stack row*. In this implementation, it is a variable-length prefix that the word does not inspect.

Types alone do not describe stack shuffles. Square-bracket identifiers record which positions carry the same symbolic value:

```
DUP   ( X[1] -- X[1] X[1] )
SWAP  ( X[2] X[1] -- X[1] X[2] )
OVER  ( X[2] X[1] -- X[2] X[1] X[2] )
DROP  ( X -- )
```

Equal identifiers mean that the positions refer to the same abstract input. Different identifiers do not assert that concrete values are unequal. An unindexed occurrence is treated as a fresh symbolic cell. Before a word effect is used, its identifiers are renamed so that separate word calls cannot accidentally share variables.

The bracket notation is explicit internal notation. Standard Forth prose normally writes numeric suffixes, such as `n1 n2 - n3`. It omits the suffix when all displayed items of that type denote one value, as in `DUP ( x - x x )`. Section 4 returns to output formatting.

3 Checking Code Sequences

Suppose a checked prefix has one effect and the next word has another. The checker compares the first effect's output with the next word's input, starting at the top of stack and moving toward deeper cells.

The implementation can be described by the following pseudocode. Here `replaceEverywhere` updates both input and output sides of both working effects.

```
compose(first, second):
    rename identifiers so the effects use disjoint names

    while first has an output and second has an input:
        actual   = top output of first
        expected = top input of second

        if actual.type and expected.type are unrelated:
            return TYPE_CLASH

        mergedType = the more specific boundary type
        mergedId   = a fresh identifier
        replaceEverywhere(actual, expected,
                          mergedType, mergedId)
        remove the matched output and input

    preserve any unmatched input requirement
    preserve any unmatched output result
    return the combined effect
```

The global replacement step is important. If a boundary match refines an `X` to `n`, every copy of that symbolic value must become `n`. Updating only the two boundary positions would lose information carried through `DUP`, `SWAP`, or `OVER`.

When the first output becomes empty before the second input, the remaining requirements of the second word become additional requirements of the whole phrase. They are placed below the requirements already inferred for the first phrase. When the second input becomes empty first, the unmatched output of the first phrase is retained below the second word's results. This states the base cases directly and avoids ambiguous primed variables in the recursive definition.

3.1 Example: `1 2 + DUP`

The individual effects are:

```
1    literal integer  ( -- n )
2    literal integer  ( -- n )
+                     ( n n -- n )
DUP                   ( X[1] -- X[1] X[1] )
```

The reduction proceeds in four steps:

1. `1` leaves one fresh `n`.
2. `2` leaves a second fresh `n` above it.
3. `+` consumes both numbers and leaves one new `n`.
4. `DUP` accepts any `X`; matching refines that `X` to `n`, and replacement updates both copies.

The inferred phrase effect is:

```
( -- n[1] n[1] )
```

The native trace exposes the same steps:

```
1    ( -- n )
2    ( -- n )
+    ( n n -- n[1] )
DUP  ( n[1] -- n[1] n[1] )
< n[1] n[1]
```

For fixed identifier-renaming rules, this algorithm is deterministic. Another implementation may choose different identifier numbers, but consistent renaming does not change the data-flow relationships.

4 Internal Identities

Internal analysis and user-facing formatting have different requirements. The analyzer should retain every known identity relationship. The printer should use consecutive suffixes in the standard Forth style.

For example, the internal effect

```
( x[7] x[8] -- x[7] x[9] )
```

can be presented as:

```
( x1 x2 -- x1 x3 )
```

This notation shows that the deeper output cell is the first input (`x1`). By contrast, `( x x - x x )` conceals that relationship.

The current artifact retains a visible identifier in either of two cases. The identifier must have been written explicitly in a source specification, or its identity must occur more than twice in the working data. An automatically created identity occurring once or twice may be printed without an identifier. The implementation also stores the abbreviated effect for subsequent use. Consequently, formatting can remove information required by a later composition.

This policy provides no analytical benefit. It provides only concise, backward-compatible output. A revised implementation should:

1. retain all identity classes in dictionary entries;
2. renumber them consecutively when an effect is displayed;
3. omit suffixes only when the Forth convention remains unambiguous.

5 Branches and Control-Flow Joins

A branch creates two possible stack states. Code after the branch needs one interface. Matching stack depth is not sufficient: one arm may leave an address where the other leaves a number, or the arms may preserve different input values.

A checker can retain path-specific effects or compute a conservative effect that discards selected facts. It can instead require annotations or reject a program when no common interface can be constructed. The current checker uses a partial variance-aware operation named `glb` in the source. Earlier order-theoretic work motivated this name. The algorithm below, rather than lattice terminology alone, defines the implemented operation.

5.1 Equal visible depths

When both effects have the same number of inputs and outputs, the checker aligns corresponding positions, but inputs and outputs have different variance. An aligned input pair keeps the more specific comparable type, because either arm must be executable. An aligned output pair keeps the more general comparable type, because code after the branch may rely only on the guarantee made by both arms. For output identities, the checker compares the ordered pair of source identities contributed by the two arms; two merged outputs share an identity only when both source pairs agree. Thus branch meeting does not invent value equality.

For example, these two store-like effects can be combined:

```
( X a-addr -- )
( X c-addr -- )
```

The common interface requires `a-addr`, because it is the more specific address type accepted by both declarations. Output variance goes the other way. For `@ ( a-addr - X )` versus `C@ ( c-addr - char )`, the common effect is `( a-addr - X )`: both branches accept an `a-addr`, but only the supertype `X` is guaranteed afterward. The declared effect `IF ( flag - )` is composed separately and accounts for the selector.

5.2 Unequal visible depths

Effects of different displayed depths can still describe compatible behavior because each effect omits an unchanged deeper prefix. Let the first effect show `k` more inputs than the second. A meet is possible only if it also shows exactly `k` more outputs. The shorter effect is interpreted as

preserving an implicit `k`-cell deep prefix. Its active input and output suffixes are aligned at offset `k` with the longer effect; for the extra output positions, the longer arm's outputs are compared with the corresponding implicit input cells preserved by the shorter arm.

For example:

```
longer:   ( X[1] n -- X[1] char )
shorter:  (      n --      char )
```

The longer effect makes one cell of the implicit prefix visible. Both arms preserve that cell, and their remaining `n - char` behavior agrees, so the common interface is the longer effect.

The following pair is incompatible:

```
( X -- X X )
(   --     )
```

The input-depth difference is one, whereas the output-depth difference is two. Consequently, `IF DUP THEN`, without a matching `ELSE`, is rejected because only one runtime path leaves the additional item.

5.3 The `MAYSWAP` example

Consider:

```
: MAYSWAP  IF SWAP THEN ;
```

The true arm swaps two cells; the false arm preserves them. A standard single-effect summary that makes no unsupported identity claim is:

```
( X1 X2 flag -- X3 X4 )
```

A more precise path-sensitive analysis would retain two alternatives: the outputs are either `X2 X1` or `X1 X2`. The implemented meet instead discards the branch-dependent permutation and reports the conservative single contract:

```
( X X flag -- X X )
```

No unsupported identity is claimed. This illustrates the output-correlation rule: an identity survives only when both arms establish the same source pair. The single-effect domain necessarily loses the path-specific permutation; a multiple-effect representation would preserve it.

6 Loop Invariants

A loop introduces a back edge. The stack state produced by one pass must be valid at the next pass, and the state reported after the loop must cover every possible exit.

A conventional data-flow analyzer maintains an abstract stack at the loop header. It propagates the stack through the body and merges the result at the header. The analysis repeats until the state stabilizes and thereby computes a fixed point [4]. Another design requires a declared loop interface and checks every back edge against it.

The current evaluator instead uses a bounded local test:

```
onePass = effect(body)
twoPass = compose(effect(body), effect(body))
summary = commonEffect(onePass, twoPass)
```

This test has constant iteration depth. It detects every nonzero net change in stack depth. If one pass changes the depth by one cell, two passes change it by two; therefore, their interfaces

cannot agree. The test does not establish that the returned identity relationships hold after every number of passes.

6.1 Counterexample

The limitation can be shown with one data-cell type and two synthetic words:

```
P      ( X[1] X[2] -- X[3] X[1] )
NEXT?  ( -- flag )
UNTIL  ( flag -- )
```

`NEXT?` supplies a termination flag. It is immediately consumed by `UNTIL`, so one complete loop pass has the data-stack effect of `P`.

Using symbolic values, repeated `P` executions behave as follows:

```
input:         A B
one pass:      C A
 two passes:   D C
three passes:  E D
```

Only one pass preserves the original lower input in the top result. The two-pass and three-pass effects contain no output tied to either original input. Nevertheless, the current join can identify an unrelated two-pass variable with the original input. It accepts:

```
: TEST  BEGIN P NEXT? UNTIL ;
```

and reports:

```
TEST ( X[2] X[1] -- X[3] X[2] )
```

This report is equivalent, after renaming, to the one-pass relationship. Composing the reported summary with itself loses that relationship, so the summary is not stable. A checker requiring an inductive loop interface would reject this candidate or weaken it to an interface with no input/output identity claim.

This counterexample identifies the precise limitation. The one-versus-two rule detects depth changes, but it does not prove that the returned typed data-flow summary covers arbitrary iteration counts.

7 Processing Forth Source

Stack-effect composition alone cannot process Forth source. Some words consume the next name, scan to a delimiter, start a definition, end a definition, or execute immediately during compilation. The specification format records these roles:

```
"("      parse until ")"          ( -- )
CONSTANT parse word define        ( X -- )
VARIABLE parse word define        ( -- a-addr )
:        parse word define colon  ( -- )
;        control end              ( -- )
IF       control if               ( flag -- )
literal integer                   ( -- n )
```

Ordinary comments are consumed but are not trusted as type declarations. Thus a source comment such as `( - n )` cannot override an inferred effect. A recognized locals declaration may provide a provisional definition shape, but that is a separate parser feature.

7.1 Declarative control structures

Control syntax can be described by a source pattern and an effect recipe. The example profile uses `FI` as a synonym for the standard `THEN` spelling:

```
syntax:
  IF <then branch> [ELSE <else branch>] FI
effect:
  IF
  either <then branch> <else branch>
```

The recipe language provides three principal operations. It sequences captured regions, combines two alternatives, and applies the one-versus-two test to a repeated region. Bare control words contribute their declared runtime data-stack effects. Parser behavior and data-stack behavior therefore remain separate from the combination algorithms.

7.2 The compile-time control-flow stack

A control word has both a runtime role and a compilation role. At runtime, `IF` consumes a flag. During compilation, the Forth standard describes `IF` as producing an `orig` item on the control-flow stack; `THEN` resolves that item [6].

The current checker records `( flag - )` as the runtime data-stack effect and records `control if` as parser metadata. Structural parsing catches missing or misnested delimiters, but it does not represent the control-flow stack as a second typed stack. A future extension could assign compile-time effects over items such as `orig`, `dest`, and `do-sys`, independently of runtime data-stack effects.

8 Complete Example

The bundled `real` profile consists of:

- `ex1types.txt`, containing types and subtype edges;
- `ex1specs.txt`, containing word and parser specifications;
- `ex1prog.txt`, containing the checked source.

Run it from the artifact root with:

```
./run-evaluator-gforth.sh real
```

8.1 Representative profile entries

The type declarations include `X`, `n`, `char`, `flag`, `addr`, `c-addr`, and `a-addr`. Relevant word declarations are:

```
DUP      ( X[1] -- X[1] X[1] )
SWAP     ( X[2] X[1] -- X[1] X[2] )
literal integer ( -- n )
IF       control if ( flag -- )
ELSE     control else ( -- )
FI       control fi ( -- )
BEGIN    control begin ( -- )
WHILE    control while ( flag -- )
REPEAT   control repeat ( -- )
```

```
UNTIL    control until ( flag -- )
DP       ( -- a-addr )
@        ( a-addr -- X )
C@       ( c-addr -- char )
0=       ( n -- flag )
```

`DP` is a system-specific example word for the data-space or dictionary pointer. The checker does not assign this effect implicitly. The profile explicitly declares `( - a-addr )`.

The current specification dictionary accepts one effect per word. It cannot directly declare both `0= ( n - flag )` and `0= ( u - flag )`. A profile can instead specify a shared accepted type. For example, the bundled Forth-2012-like profile declares `0= ( x - flag )`. This effect accepts both signed and unsigned cells. A stricter profile can introduce an `integer` supertype above `n` and `u`. Direct support for multiple effects or overload selection would preserve greater precision.

8.2 Program and output

The program defines five words and checks one top-level phrase:

```
: PEEK2 DUP @ SWAP @ ;
: MAYSWAP IF SWAP FI ;
: NLOOP BEGIN DUP 0= WHILE REPEAT ;
: ULOOP BEGIN DUP 0= UNTIL ;
: KEEPIF IF DUP ELSE DUP FI ;

DP DP C@ 0= KEEPIF
```

The exact native log contains:

```
PEEK2   ( a-addr -- X X )
MAYSWAP ( X X flag -- X X )
NLOOP   ( n[1] -- n[1] )
ULOOP   ( n[1] -- n[1] )
KEEPIF  ( X[1] flag -- X[1] X[1] )
```

The `MAYSWAP` line illustrates conservative information loss: the single summary retains stack shape and types but does not choose one of the two path-specific permutations. Section 5 gives the variance-aware rule that produces this contract.

The top-level trace ends with:

```
DP      ( -- a-addr[1] )
DP      ( -- a-addr )
C@      ( a-addr -- char )
0=      ( char -- flag )
KEEPIF  ( a-addr[1] flag -- a-addr[1] a-addr[1] )
< a-addr[1] a-addr[1]
```

`C@` requires `c-addr`; `DP` provides the more specific `a-addr`, so matching succeeds. In this profile `char` is a subtype of `n`, so `0=` accepts the result. `KEEPIF` consumes the flag and duplicates the surviving address on either branch.

9 Computing Process

After one non-recursive source body has been parsed, its evaluation is a post-order traversal of a finite syntax tree:

- a leaf obtains an effect from word lookup or literal classification;

- a sequence node invokes straight-line composition;
- a branch node invokes the common-effect operation;
- a repetition node invokes the one-versus-two operation.

Every node depends only on its children, and each implemented operation is deterministic. The parsed body therefore has one implementation result: an effect or a clash. This property characterizes the program executed by the checker; it is not a type-soundness theorem. In particular, it does not show that primitive specifications agree with runtime behavior. It also excludes unrestricted recursion, dynamic execution, and some forms of dictionary construction.

The native implementation is the single source file `gforth-evaluator.fs`. Its main components are organized by prefix:

- `ev-ts-*`: types, aliases, and subtype relationships;
- `ev-sym-*`: typed cells and symbolic identifiers;
- `ev-spec-*`: stack effects and the three combination algorithms;
- `ev-ss-*`: specification dictionary;
- `ev-sc-*` and `ev-parse-*`: scanning and source parsing;
- `ev-diag-*` and `ev-log-*`: diagnostics and logs.

Effects are mutable records. Dictionary effects are deep-cloned before use, independent branches receive disjoint identifier ranges, and input line endings are normalized. These details prevent one analysis from changing a later dictionary lookup.

Boundary matching is linear in the shorter touching boundary, but every match currently substitutes through complete working vectors. Large intermediate effects can therefore make reduction roughly quadratic in effect size. A union–find structure for identifier classes would reduce repeated copying.

10 Artifact and Evaluation

10.1 Artifact manifest

The files required for the paper example are included explicitly. The main artifact paths are:

- `gforth-evaluator.fs`: native checker;
- `run-evaluator-gforth.sh` and `run-evaluator-gforth.bat`: launchers;
- `ex1types.txt`, `ex1specs.txt`, and `ex1prog.txt`: the example in Section 8;
- `forth2012types.txt`, `forth2012specs.txt`, and `forth2012prog.txt`: the larger Forth-2012-like profile;
- `testdata/positive` and `testdata/negative`: regression programs;
- `README.md` and `EVALUATOR.md`: formats and commands.

The launcher creates `ex1prog.txt.log`; the log is output, not a required input. An archive containing only the Forth-2012 files cannot reproduce the `ex1` example.

10.2 Regression results

The checked-in corpus contains 29 programs: 13 expected accepts and 16 expected rejects. A current native run produces the following results:

Corpus class	Programs	Expected	Result
Forth-2012 positive	7	accept	7 accepted
Custom-profile positive	5	accept	5 accepted
Article positive	1	accept	1 accepted
Negative	16	reject	16 rejected

The Forth-2012 environment contains 50 lines of type declarations and 278 lines of vocabulary and control specifications. The native checker is 4,483 lines of Gforth. These counts document the artifact's scale but do not establish soundness or performance. A regression corpus cannot reveal an error shared by the checker, its profile, and the expected result.

11 Related Work

Forth stack effects. The Forth standard uses stack comments and numeric suffixes but does not make every comment a static judgment [6, 14]. Earlier work by the present author developed algebraic composition, multiple effects, typed wildcards, inheritance, and must-analysis operators [8, 9, 10, 11, 12]. The current artifact combines a subset of those mechanisms with source parsing, external control declarations, inferred definitions, and diagnostics. Its input format is less expressive than the earlier multiple-effect work because it currently accepts one effect per word.

StrongForth. StrongForth, developed by Stephan Becher, integrates static typing into a Forth system. Recent articles cover object-oriented typing, programming practice, and word overloading [1, 2, 3]. Overloading is relevant to words such as `0=` and `EXECUTE`, where one broad common type loses distinctions. StrongForth targets a typed Forth dialect; the present checker leaves the target system unchanged and applies an external profile. StrongForth therefore demonstrates that limitations of the current artifact are not inherent limitations of Forth syntax.

Optional and pluggable Forth typing. Riegler's thesis presents a Haskell prototype with optional checkers ranging from stack underflow and overflow checks to stronger static type consistency [15]. It covers subtyping, control structures, reference types, multiple stack effects, compile-time programming, object orientation, and higher-order programming. The present artifact adds a native Gforth implementation, external parser and control profiles, and source-oriented traces, but is narrower in multiple effects, compile-time stack typing, and higher-order features.

Other stack-machine type systems. Stoddart and Knaggs study type inference for stack-based languages [17]. Saabas and Uustalu present compositional systems for stack-based low-level code [16]. These works assume more restricted instruction languages than Forth source. They therefore do not model an extensible outer interpreter in which words consume source text or create definitions. A fixed instruction set facilitates conventional typing rules and soundness arguments.

WebAssembly. The online WebAssembly Core Specification identifies itself as Release 3.0 [18]. WebAssembly does not infer a loop interface by comparing one and two executions of a body. A block type declares the operand types consumed and produced. A control stack records each active block's label type and stack height. Every branch is checked against its target label. A

loop back edge therefore targets a declared loop-label input. WebAssembly also defines controlled stack polymorphism for unreachable code and has mechanized metatheory [19, 7].

This comparison locates the trade-off. WebAssembly obtains stronger branch and loop guarantees from fixed control syntax and explicit block interfaces. The present Forth checker infers interfaces from source while loading parser and control behavior as data. This strategy supports retargeting and source-level Forth analysis. Its limitations include lossy normalization, a single variance-aware branch contract rather than path alternatives, and the one-versus-two loop rule.

Higher-order concatenative code. A generic stack-prefix parameter stands for any unchanged sequence below a word's visible operands. A first-class quotation is a value that itself has a stack transformation. Some higher-order words require a quotation that works for every possible stack prefix. In type-system terminology, the generic parameter then appears inside a function argument type; this is often called higher-rank, or rank-2, polymorphism. The term describes where the generic parameter is introduced, not the number of stack cells.

A recent account of the IV language avoids automatic inference of these nested generic types by requiring definition annotations and explicit parameters for execution and composition operators [5]. The current checker has an implicit unchanged prefix but no quotation type and no general algorithm for prefix variables. Supporting first-class quotations therefore requires an extension of the type language, not another primitive word entry.

12 Limitations and Next Steps

The current implementation has the following known limits:

- Primitive and parser specifications are trusted. An incorrect declaration can make a program appear valid.
- Only equal or subtype-related boundary types match; one word cannot yet have several alternative primitive effects.
- Normalization can remove identity relationships that later checks need.
- Branch joining keeps one variance-aware common contract and therefore loses path-specific facts such as the two possible `MAYSWAP` permutations.
- Loop checking compares one pass with two and does not compute an inductive header state.
- Compile-time control-flow items are parsed structurally but are not represented as typed stack effects.
- Return-stack manipulation, unrestricted `EXECUTE`, non-local `THROW`, self-modifying definitions, and unrestricted metaprogramming require additional models.
- Recursive dictionary construction and general forward references are outside the deterministic parsed-body characterization.

The immediate engineering priorities are to preserve identity classes internally and adopt standard-style output. Subsequent extensions can add multiple primitive effects, typed compile-time control items, explicit stack-prefix and quotation types, branch alternatives, and fixed-point loop analysis. A semantic soundness result also requires an operational model for trusted primitives and precise rules for input and output constraints.

13 Conclusion

This work describes a configurable source-level checker for Forth rather than a general type system for every concatenative language. Its core mechanisms are subtype-aware boundary matching, symbolic value identifiers, and external profiles for Forth words and parser behavior. These mechanisms check straight-line source and provide reproducible inference for non-recursive definitions.

Branches and loops require more than stack-depth equality. The current branch operator computes a variance-aware common contract, while the one-versus-two loop test remains an experimental approximation. Neither replaces path-sensitive control-flow analysis or inductive invariants. The `MAYSWAP` and `P` examples demonstrate the information lost by those compact summaries.

The artifact provides executable stack-effect documentation and a platform for experiments with different Forth profiles. Stronger guarantees require the preservation of internal identities, alternative effects, typed compile-time control items, and fixed-point loop analysis.

References

[1] S. Becher. Statische Typprüfung und Objektorientierung. *Forth–Magazin Vierte Dimension*, 41(1):11–15, 2025.
`https://forth-ev.de/wiki/res/lib/exe/fetch.php/vd-archiv:4d2025-01.pdf`.

[2] S. Becher. Statische Typprüfung: Programmierpraxis. *Forth–Magazin Vierte Dimension*, 41(2):15–20, 2025.
`https://forth-ev.de/wiki/res/lib/exe/fetch.php/vd-archiv:4d2025-02.pdf`.

[3] S. Becher. Statische Typprüfung: Überladung von Wörtern. *Forth–Magazin Vierte Dimension*, 41(3):14–17, 2025.
`https://forth-ev.de/wiki/res/lib/exe/fetch.php/vd-archiv:4d2025-03.pdf`.

[4] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, pp. 238–252, 1977. doi:10.1145/512950.512973.

[5] A. M. Dinikeev. Type system for a statically typed concatenative programming language with first class function support. *Information-measuring and Control Systems*, no. 5, pp. 15–25, 2025. doi:10.18127/j20700814-202505-02.

[6] Forth 200x Standardisation Committee. *Forth 2012 Standard*. Draft proposed standard, 10 November 2014. `https://forth-standard.org/standard/intro`.

[7] D. McDermott, Y. Morita, and T. Uustalu. A type system with subtyping for WebAssembly's stack polymorphism. In *ICTAC 2022*, LNCS 13572, pp. 305–323, 2022. doi:10.1007/978-3-031-17715-6_20.

[8] J. Pöial. Algebraic specifications of stack effects for Forth programs. In *1990 FORML Conference Proceedings, EuroFORML'90*, pp. 282–290, 1991.

[9] J. Pöial. Multiple stack-effects of Forth programs. In *1991 FORML Conference Proceedings, euroFORML'91*, pp. 400–406, 1992.

[10] J. Pöial. Algebraic specifications of stack effects. *Forth Dimensions*, 16(4):18–20, November/December 1994. `https://www.forth.org/fd/FD-V16N4.pdf`.

[11] J. Pöial. Stack effect calculus with typed wildcards, polymorphism and inheritance. Abstract in *18th EuroForth Conference*, p. 38, 2002; accompanying presentation slides.
`https://www.complang.tuwien.ac.at/anton/euroforth/ef02/poial02.pdf`.

[12] J. Pöial. Typing tools for typeless stack languages. In *Proceedings of the 22nd EuroForth Conference*, Cambridge, England, pp. 40–46, 2006. https://www.complang.tuwien.ac.at/anton/euroforth/ef06/poial06.pdf.

[13] J. Pöial. Java framework for static analysis of Forth programs. In *Proceedings of the 24th EuroForth Conference*, Vienna, Austria, pp. 20–23, 2008. https://www.complang.tuwien.ac.at/anton/euroforth/ef08/papers/poial.pdf.

[14] E. D. Rather, D. R. Colburn, and C. H. Moore. The evolution of Forth. In *History of Programming Languages—II*, pp. 625–670, ACM, 1996. doi:10.1145/234286.1057832.

[15] G. Riegler. *Evaluation and Implementation of an Optional, Pluggable Type System for Forth.* Diploma thesis, Technische Universität Wien, 2015. doi:10.34726/hss.2015.27899. https://repositum.tuwien.at/handle/20.500.12708/7117.

[16] A. Saabas and T. Uustalu. Compositional type systems for stack-based low-level languages. In J. Gudmundsson and C. B. Jay, editors, *Proceedings of CATS 2006*, CRPIT 51, pp. 27–39. Australian Computer Society, 2006. https://cs.ioc.ee/~tarmo/papers/saabas-uustalu-cats06.pdf.

[17] B. Stoddart and P. J. Knaggs. Type inference in stack based languages. *Formal Aspects of Computing*, 5(4):289–298, 1993. doi:10.1007/BF01212404.

[18] WebAssembly Community Group. *WebAssembly Core Specification, Release 3.0.* Editor: A. Rossberg; online specification. https://webassembly.github.io/spec/core/.

[19] C. Watt. Mechanising and verifying the WebAssembly specification. In *CPP 2018*, pp. 53–65, ACM, 2018. doi:10.1145/3167082.